

s1.08 — Scale a monolithic service; identify components to separate

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Migration and rollback — IC3 baseline](#)
- [Outbox relay and idempotent execution — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: retain the first-hand report attribution in [the bank](#).
 - The architecture below is a proposed interview answer.
 - It does not claim to reproduce Oracle's implementation or an interviewer's accepted answer.
- Why this is hard
 - Naive: split the monolith into services along the team chart and put a gateway in front.
 - First break: the database was the bottleneck, so the same queries now arrive over more network hops with no shared transaction, and p99 gets worse.
 - Second break: write ownership moves during migration, both executors publish results for the same job, and rollback is impossible once only the new store holds recent writes.
 - Minimum primitives
 - A measured profile
 - One routing seam
 - A transactional outbox
 - A single authoritative writer per entity
 - A staged cutover with a rollback gate.
- 🗨️ "A process split changes failure and ownership boundaries; it does not remove a bottleneck you have not measured."
- Opening script
 - "I will start with diagnosis, because a split that does not target the measured bottleneck only adds hops."
 - "I assume a transactional API with expensive report generation running in the request process against one relational database."
 - "The first extraction is derived execution, the reports, because it buys independent compute without splitting transactional ownership."

- “The contract is a 202 job API backed by a transactional outbox, so accepted work survives a queue outage.”
- “The deep dive is the staged migration and the exact point after which rollback needs data movement.”
- “I will close with failure modes, admission control, and when full domain extraction is actually justified.”

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Symptom, assumed system, ownership invariant
Estimates and contract	11:00–17:00	CPU, concurrency, report backlog; job API semantics
Architecture	17:00–30:00	Diagram, extraction boundary, one job trace
Deep dive	30:00–45:00	Staged migration, rollback gate, outbox and idempotent execution
Operations and tradeoffs	45:00–50:00	Failure table, admission control, tradeoff, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
What is the symptom?	Interactive p99 breaches 300 ms during report-heavy periods	Points at coupling between report CPU and request threads, not raw throughput
Where is the bottleneck?	Request-process CPU from reports; database is near but under capacity	Extraction targets compute; if the database were saturated, workers would only move the problem
Which domain?	Order management with on-demand reports; the prompt names none	Gives a concrete authoritative entity and a derived output to separate
Must data ownership split?	No; one relational database stays the transactional owner	Rollback stays a routing change; no distributed transactions
Multi-tenant?	Yes; tenant context is explicit on every request and job	Idempotency keys, quotas, and authorization are tenant-scoped
Deployment coupling?	Reports and interactive code ship in one binary today	Extraction also buys independent deploys, which counts in the decision
External side effects in reports?	None beyond writing a result object	Duplicate execution is wasteful, not harmful; publication decides the winner

- Functional scope
 - Preserve the public synchronous API and its semantics.
 - Submit reports as asynchronous jobs with a stable ID and a status resource.
 - Execute reports in an isolated worker pool with bounded admission.
 - Migrate execution in stages with parity checks and a defined rollback.
 - Keep authorization and tenant context intact across the new seam.
- Assumed system
 - A transactional application with synchronous API traffic.
 - Expensive report generation currently runs in the request process.
 - One relational database owns business transactions.
 - No requirement to split every domain into a service.

Requirement	Target	Enforced by
Peak interactive throughput	5,000 requests/s	Horizontal app tier behind the facade; bounded pools
Interactive latency	p99 < 300 ms	Reports removed from request threads; per-endpoint SLO alarms
Report submission latency	p99 < 200 ms	One transaction writes job plus outbox; no execution inline
Report completion	p99 < 60 s under admitted load	Worker pool sized above arrival rate; admission cap on backlog
Interactive availability	99.95%	Queue and workers are off the interactive path; facade fails open to the monolith
Migration data safety	0 committed transactional writes lost from executor migration	Database ownership unchanged; compatible schema and fenced result publication
Rollback readiness	Route new jobs back within 5 min in a rehearsed drill	Cached routing generation and fenced executor claims
Static stability	30 min routing-authority outage budget	Versioned cached assignments; no ownership changes until authority recovers

- Invariants
 - Exactly one authoritative writer owns a business entity at a time.
 - A committed report job is eventually observable and executable.
 - An event retry cannot repeat a non-idempotent business effect.
 - An authorization rule does not disappear when a path moves to a new process.
- Non-goals
 - Splitting every bounded context into a service.
 - Distributed transactions across services.
 - Rewriting the application framework or storage engine.
 - Multi-region active-active for the monolith.

Capacity and consequence — IC3 baseline

- CPU example
 - $5,000 \text{ requests/s} \times 20 \text{ ms CPU/request} = 100 \text{ CPU-seconds/s}$.
 - At a 60% target utilization, budget about 167 effective cores before resilience headroom.
 - This is a workload estimate, not a hardware benchmark.
 - If measured work is waiting on a database, extra application cores may not help.
 - Decision: horizontal app scaling is justified only if the profile shows CPU-bound handlers
 - Otherwise the money goes to the database or the query plan.
- Concurrency
 - At 200 ms average response time, Little's Law gives about 1,000 requests in flight.
 - Database connections need not equal request concurrency.
 - Size the pool against database service time and capacity.
 - Decision: cap the pool well below in-flight requests, for example 1,000 in flight against an order-of-magnitude 100 connections, and queue requests in the app tier with a deadline rather than at the database.
- Report extraction
 - Assume 1% of requests launch a 2 s CPU report.
 - $50 \text{ reports/s} \times 2 \text{ s} = 100 \text{ CPU-seconds/s}$ for reports alone.
 - An asynchronous worker pool can isolate that load from interactive requests.
 - If a pool processes only 40 reports/s, backlog grows by 10/s indefinitely.
 - Queueing requires an admission limit, not just a broker.

- Investigate first
 - Endpoint traces.
 - CPU profiles.
 - Query plans and lock waits.
 - Connection-pool wait time.
 - Tail-latency correlation with background jobs.
 - Decision: no extraction is proposed without a profile that names the resource
 - The estimate above is what the profile must confirm.

Architecture and flow — IC3 baseline

```
Migration routing table (control plane) --cached routes--> facade
```

Clients -> existing API / routing facade -> transactional monolith -> primary DB

```
      |                                     |
      |                                     report job + outbox
      |                                     |
+-> report status API                    outbox relay
                                         |
                                         durable queue
                                         |
                                         report workers
                                         |
                                         immutable results
```

- Baseline improvements before extraction
 - Remove measured inefficient queries.
 - Add justified indexes.
 - Bound connection pools and request queues.
 - Externalize session state if it prevents horizontal scaling.
 - Cache only reads with an explicit staleness contract.
- Routing facade owns the seam.
 - Mechanism: stateless routing by tenant cohort from a cached assignment
 - Public API unchanged.
 - Cost: one more hop on every request and a component that must fail open to the monolith.
- Migration routing table (control plane) owns cohort assignments.
 - Mechanism: small store of (tenant, context, cohort) → owner, epoch
 - The facade polls and caches last-known-good.
 - Cost: a route change is visible only after the poll interval
 - Static stability is bought with bounded staleness.
- Monolith remains the authoritative writer.
 - Mechanism: job creation and outbox insert commit in one database transaction.
 - Cost: job submission still consumes monolith capacity
 - Only execution moves.
- Outbox relay owns publication.
 - Mechanism: polls committed outbox rows in order, publishes, marks sent.
 - Cost: at-least-once publication
 - Every consumer must be idempotent.
- Durable queue owns buffering.
 - Mechanism: partitioned by tenant with retention longer than the worst expected worker outage.
 - Cost: visibility is not execution

- Duplicates and expired leases are normal.
 - Report workers own derived output.
 - Mechanism: claim with a lease and attempt number, write an immutable result, publish through a conditional state transition.
 - Cost: workers read the primary database, so a report surge is still a database load unless a read model exists.
 - Extraction boundary
 - Move expensive report execution first.
 - Keep report submission/status in the existing authoritative service initially.
 - Workers own derived output, not unrelated transactional tables.
 - This reduces transaction-splitting risk while buying independent compute scaling.
- Authorize report submission and validate input.
- In one database transaction, create the job and its outbox event.
- Return 202 with the stable job ID.
- Relay the committed event to the queue.
- Worker claims execution and creates output with stable job/attempt identity.
- Publish the result reference using a conditional state transition.
- Client polls or receives an authorized completion notification.

API and data model — IC3 baseline

Surface	Contract	Conflict/error behavior
Existing synchronous API	Retain current external semantics	Facade routes by cohort; no client migration
POST /v1/report-jobs	Returns 202, job ID, status URL	Tenant-scoped idempotency key plus request hash; same key with different payload returns 409
GET /v1/report-jobs/{id}	State, progress, result reference	Cross-tenant ID returns 404; progress is bounded, not exact
DELETE /v1/report-jobs/{id}	Requests cancellation	Best effort before publication; 409 once a result is published; idempotent if already cancelled
GET /v1/report-jobs	Cursor-paginated status listing	Query by tenant and creation time; cursor is opaque and tenant-bound
Operator POST /v1/migrations	Assign a cohort to an executor	Idempotent by (context, cohort, target); returns 202 with an operation ID
Operator POST /v1/migrations/{id}:rollback	Route a cohort back	409 if data compatibility no longer permits rollback

State	Key	Fields/invariant
Job	(tenant_id, job_id)	Request hash, state, version, attempt, result reference
Create dedup	(tenant_id, idempotency_key)	Same request returns same job; changed payload conflicts; retained past the retry horizon
Outbox	event_id	Job ID and committed transition; same transaction as job creation
Worker claim	(job_id, attempt)	Lease/owner and fencing version if shared writes are possible
Result	(tenant_id, job_id, attempt)	Immutable output; only winning attempt becomes visible
Ownership route	(tenant_id, context, cohort)	Owner service, routing epoch, migration state

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).

- Queue messages and their visibility timeouts live in the durable queue; SQL holds jobs, dedup, the outbox, claims, result references, and routes.

```
CREATE TABLE report_jobs (
  tenant_id text NOT NULL, job_id text NOT NULL, request_hash text NOT NULL,
  state text NOT NULL CHECK (state IN ('ACCEPTED','RUNNING','SUCCEEDED','FAILED','CANCELLED')),
  version bigint NOT NULL, -- CAS target for every transition
  attempt integer NOT NULL DEFAULT 0 CHECK (attempt >= 0),
  result_ref text, -- set only by the winning attempt's CAS
  created_at timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, job_id),
  CHECK (state <> 'SUCCEEDED' OR result_ref IS NOT NULL)
);
CREATE INDEX report_jobs_list ON report_jobs (tenant_id, created_at, job_id);
CREATE INDEX report_jobs_running ON report_jobs (tenant_id, job_id) WHERE state = 'RUNNING';
CREATE TABLE report_job_dedup (
  tenant_id text NOT NULL, idempotency_key text NOT NULL,
  request_hash text NOT NULL, job_id text NOT NULL,
  expires_at timestamptz NOT NULL, -- past the client retry horizon
  PRIMARY KEY (tenant_id, idempotency_key),
  FOREIGN KEY (tenant_id, job_id) REFERENCES report_jobs
);
CREATE INDEX report_job_dedup_expiry ON report_job_dedup (expires_at);
CREATE TABLE job_outbox (
  event_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  tenant_id text NOT NULL, job_id text NOT NULL,
  transition text NOT NULL, job_version bigint NOT NULL, -- committed in the job's transaction
  created_at timestamptz NOT NULL, sent_at timestamptz,
  FOREIGN KEY (tenant_id, job_id) REFERENCES report_jobs
);
CREATE INDEX job_outbox_unsent ON job_outbox (event_id) WHERE sent_at IS NULL;
CREATE TABLE worker_claims (
  job_id text NOT NULL, attempt integer NOT NULL CHECK (attempt > 0),
  tenant_id text NOT NULL, owner_id text NOT NULL,
  owner_epoch bigint NOT NULL, -- routing epoch; fences an executor that lost ownership on
rollback
  lease_expires_at timestamptz NOT NULL, claimed_at timestamptz NOT NULL,
  PRIMARY KEY (job_id, attempt),
  FOREIGN KEY (tenant_id, job_id) REFERENCES report_jobs
);
CREATE INDEX worker_claims_lease ON worker_claims (lease_expires_at);
CREATE TABLE job_results (
  tenant_id text NOT NULL, job_id text NOT NULL, attempt integer NOT NULL,
  object_ref text NOT NULL, size_bytes bigint NOT NULL CHECK (size_bytes >= 0),
  sha256 text NOT NULL, produced_at timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, job_id, attempt),
  FOREIGN KEY (job_id, attempt) REFERENCES worker_claims
);
CREATE TABLE ownership_routes (
  tenant_id text NOT NULL, context text NOT NULL, cohort text NOT NULL,
  owner_service text NOT NULL,
  routing_epoch bigint NOT NULL CHECK (routing_epoch >= 0), -- bumped on every assignment and rollback
migration_state text NOT NULL
  CHECK (migration_state IN ('LEGACY','SHADOW','CANARY','MIGRATED','ROLLED_BACK')),
  updated_at timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, context, cohort)
);
```

- Not captured by the DDL
 - Queue offsets, visibility timeouts, and the 24 h queue retention live in the queue; result bytes live in object storage behind `object_ref`.
 - The facade's cached last-known-good route snapshot is process memory, refreshed on the poll interval.
 - Deviation from the companion: the companion's `orders/order_lines` domain tables are replaced by the report-job tables this file's state table names; `ownership_routes` is taken as is; `worker_claims` keeps the state table's `(job_id, attempt)` key with `tenant_id` as a column.

- Conventions
 - Job state machine: `ACCEPTED` → `RUNNING` → `SUCCEEDED` | `FAILED` | `CANCELLED`
 - Transitions are compare-and-set on `version`.
 - Result URLs are signed, tenant-scoped, and short-lived.
 - Every worker call carries the verified tenant context from the job row, never from the queue message alone.

Migration and rollback — IC3 baseline

- Phase 1: establish evidence.
 - Baseline latency and error metrics by endpoint.
 - Record database load and queue-free execution behavior.
 - Define a success threshold for extraction.
- Phase 2: introduce the seam.
 - Add a routing facade or module interface.
 - Keep public API compatibility.
 - Make authorization and tenant context explicit across the seam.
- Phase 3: move derived execution.
 - Submit jobs transactionally in the current owner.
 - Shadow report generation on deterministic sample inputs.
 - Compare output hashes or domain-level totals.
 - Canary real job cohorts only after parity checks.
- Phase 4: ramp traffic.
 - Increase worker share in bounded steps.
 - Watch interactive latency, job age, and output mismatches.
 - Stop rollout when any agreed gate fails.
- Rollback
 - Route new jobs back to the previous executor.
 - Let in-flight jobs finish or cancel them through explicit state transitions.
 - Prevent both executors from publishing the same job result.
 - Preserve new schema fields until every old binary can tolerate them.
- Consistency boundary
 - Database ownership remains unchanged in this baseline.
 - This makes rollback a routing/execution change.
 - It does not require merging conflicting transactional writers.
 - Exact failure: an old executor and a new worker both hold the same job during a rollback window
 - The routing epoch on the claim makes the stale executor's publish fail its compare-and-set.

Outbox relay and idempotent execution — IC3 baseline

```
CLAIM_AND_PUBLISH(job_id, worker, epoch):
    claim = transactionally acquire runnable job with new attempt, owner epoch, and lease
    require no unexpired competing lease and a nonterminal state
    if claim fails: return (another attempt owns it or job is terminal)
    result = execute(job, attempt)          -- writes immutable result keyed by attempt
    CAS(RUNNING -> SUCCEEDED, expected attempt, owner epoch, live lease, result_ref)
    if CAS fails: leave result unreferenced for cleanup
```

- Queue visibility is not exactly-once execution.
 - Workers can receive duplicates.
 - A lease can expire while a paused worker later resumes.
 - Conditional publication selects one winning result.
 - Non-idempotent external effects need their own effect identity and contract.

- Outbox relay can publish twice after a crash.
 - Consumers still need idempotency.
 - See [transactional outbox guidance](#).
- Concrete values
 - Relay poll interval `100 ms`
 - Outbox age alarm at `30 s`.
 - Proposed lease is `4 s` for a workload with `2 s` p99 report time, renewed every `2 s`.
 - Tail pauses can exceed that lease, so expiry reclaim and result fencing remain mandatory.
 - Reclaim in one transaction that advances the attempt and persists retry dispatch intent.
 - Queue retention `24 h`, longer than the worst expected worker outage.
- Exact failure
 - A worker pauses past its lease, a second worker completes the job, then the first resumes and writes its result.
 - The first worker's publish carries a stale attempt and fails the compare-and-set
 - Its result object is orphaned and reclaimed.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Queue unavailable	New report execution	Outbox age	Interactive work continues; job remains accepted/pending	Relay after queue recovery within retention limits
Workers unavailable	Report cohort	Oldest job age	Bound new submissions; status remains readable	Restore workers and drain backlog fairly
Duplicate delivery	One job	Duplicate identity/claim	Reuse or discard redundant attempt	Publish only a valid winning attempt
Report queries overload DB	Transactional workload	DB I/O and lock waits	Throttle report reads	Use a replica/snapshot only with accepted staleness
New executor produces wrong output	Canary cohort	Parity/reconciliation	Stop new routing	Roll back and repair affected results
Partial rollout with old schema	A version cohort	Version-specific errors	Keep compatible fields and responses	Expand-contract migration
Retry storm	Shared dependencies	Retry/original request ratio	Backoff, deadlines, and load shedding	Restore capacity before increasing retries
Migration routing table (control plane) unavailable	Route changes only	Facade poll failures	Facade routes on cached last-known-good assignments	Refresh routes; reconcile any cohort moved during the outage
Primary DB failover	All writes, in-flight jobs	Connection errors, replication lag	Reads may serve from replica with stated staleness; submissions fail fast	Reconnect; relay resumes from committed outbox; leases expire and jobs re-claim

- Observe
 - Interactive p99 separately from job completion p99.
 - Queue age, not just queue depth.
 - Outbox age and duplicate rate.
 - Database resource usage by caller class.
 - Output mismatch/reconciliation counts.
- Retry discipline
 - Clients retry submission with the same idempotency key under backoff and jitter
 - The request hash catches payload drift.
 - Workers retry transient execution failures up to a bounded attempt count

- The job then lands in `FAILED` with a reason, not an infinite loop.
- The relay retries publication indefinitely with backoff, because the outbox row is the source of truth and duplicates are tolerated downstream.
- Tenant isolation
 - Pass verified tenant context to workers.
 - Per-tenant admission caps and queue shares keep one tenant's report burst from starving others.
 - Workers use least-privilege access to report inputs and result storage.
 - Signed result URLs remain scoped and short-lived.
 - Never rely solely on the facade for authorization if a worker API is independently reachable.
- Rollout and rollback safety
 - Schema changes follow expand, migrate, contract
 - Old binaries must tolerate new columns before contract.
 - Route one canary cohort at a time
 - The routing epoch on claims fences a stale executor after rollback.
 - Rollback is a control-plane route change with no data movement while ownership stays in the monolith.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Tune modular monolith	Improve code/queries in one deployment	Low operational complexity	Shared scaling/deployment	Bottleneck is local and ownership is clear
Horizontal app scaling	More stateless instances	Simple compute expansion	Database may remain bottleneck	CPU-bound request handling
Async worker extraction	Move long-running execution	Protects interactive latency	Queue and retry semantics	Baseline expensive report workload
Read model/replica	Separate derived reads	Isolates read cost	Staleness and replication lag	Reports tolerate a stated snapshot delay
Full domain extraction	Move API and data ownership	Independent evolution	Distributed transactions and migration	Boundary is stable and ownership benefits justify cost

- IC4 stretch — extract transactional ownership only when justified.
 - Backfill the new store from a consistent snapshot.
 - Capture subsequent changes with ordered source versions.
 - Reconcile counts and business invariants.
 - Stop or fence old writes at cutover.
 - Route to the new authoritative writer.
 - A read-only shadow does not prove write-path correctness.
- IC4 stretch — rollback after new writes
 - Routing back is insufficient if only the new store has recent committed writes.
 - Require reverse replication/reconciliation or a planned forward-repair strategy.
 - Declare the point after which rollback needs data movement.
- IC4 stretch — 10× load
 - 50,000 requests/s implies about 1,670 cores for interactive work and 500 reports/s
 - The single primary becomes the bottleneck before the app tier does.
 - Shard the transactional store by tenant and give reports a replica or snapshot read model with a stated freshness.
 - Partition the queue by tenant so one tenant's backlog cannot delay another's.
- IC4 stretch — cost and blast radius
 - Ten serial independent 99.9% dependencies would yield roughly $0.999^{10} \approx 99.0\%$ path availability.
 - Independence is an illustrative assumption

- Real failures can be correlated.
- Avoid splitting one request into unnecessary synchronous hops.
- Cell the facade and worker pools per cohort so a bad deploy strands one cohort, not every tenant.
- Worker cores cost the same as monolith cores
- The saving is tail-latency isolation and independent deploys, not raw CPU.
- Weakest assumption
 - Report CPU, not the database, is the measured bottleneck
 - If the database is saturated, workers only move the same queries to another process.
 - Mitigation: profile before extraction and shadow the workers against a replica to see whether database load, not CPU, is what drops.
- Over-engineering warning
 - “Microservices make it scalable” is not a diagnosis.
 - Do not split write ownership to solve a compute problem.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Symptom, assumed bottleneck, one authoritative writer	Names deployment coupling and non-goals as part of the decision	Negotiates the split instead of assuming it
Estimates	167 cores, 1,000 in flight, reports equal the interactive CPU budget	Uses the 40/s versus 50/s gap to demand admission control, not more brokers	A number changes a decision
Architecture	Facade, outbox, queue, workers, unchanged database ownership	States the cost of each hop and what static stability the facade needs	Every component has a mechanism and a price
Deep dive	Four-phase migration with parity and a rollback gate	Routing-epoch fencing; the point after which rollback needs data movement	Reasons about two executors racing on one job
Operations	Failure table, queue age, outbox age, parity counts	Routing-table outage as static stability; cohort cells; DB failover path	Volunteers degradation and rollout before being asked
Wrap	Recap and tradeoff	Names the bottleneck assumption first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: Why not split every component immediately?
 - **IC3 baseline**
 - Measure the bottleneck and extract a bounded expensive capability.
 - **IC4 stretch**
 - Include deployment ownership and cross-team coordination cost in the decision.
- Q2: The database is the bottleneck
 - Will workers solve it?
 - **IC3 baseline**
 - Not if workers issue the same expensive queries concurrently.
 - **IC4 stretch**
 - Use workload isolation, indexing, or a derived read model under a stated freshness contract.
- Q3: The job row commits but queue publication fails.
 - **IC3 baseline**
 - A transactional outbox preserves the intent for later publication.
 - **IC4 stretch**
 - Relay duplicates require idempotent consumer effects and reconciliation.
- Q4: We now require strict atomic updates across two extracted services.
 - **IC3 baseline**

- Reconsider the boundary or keep those writes in one owner.
- **IC4 stretch**
 - Only adopt distributed coordination or compensation after defining the exact invariant.
- Q5: Can you roll back after moving write ownership?
 - **IC3 baseline**
 - Only if the old owner has the required current data and is safely reactivated.
 - **IC4 stretch**
 - Define reverse replication or forward repair before cutover.
- Q6: Why does a durable queue still need admission control?
 - **IC3 baseline**
 - Arrival above service rate makes backlog and completion time grow without bound.
 - **IC4 stretch**
 - Allocate per-tenant shares and shed expensive jobs before violating global latency targets.
- Q7 (premise reversal): Why split at all instead of buying a bigger box?
 - **IC3 baseline**
 - Vertical scaling is the first move when headroom exists, but it leaves report CPU on the request threads, so the tail-latency coupling remains.
 - **IC4 stretch**
 - The extraction buys isolation and independent deploys, not raw CPU
 - Compare the cost of a queue and workers against the cost of the bigger box and its ceiling.
- Q8: The facade is a new single point of failure.
 - **IC3 baseline**
 - It is stateless and fails open to the monolith, so its outage costs routing flexibility, not availability.
 - **IC4 stretch**
 - Cache routes as last-known-good so a routing-table outage changes nothing on the data path.
- Validation scenarios
 - Submit timeout followed by same-key retry.
 - Relay crash after publication.
 - Duplicate worker claim.
 - Old and new executor overlap during rollback.
 - Schema compatibility across mixed binaries.
 - Report read surge while interactive traffic peaks.
 - Routing table unavailable during a cohort move.
 - Primary failover with a job mid-execution.

Recall card

- "Diagnose first, then isolate the measured bottleneck."
- "Move derived execution before splitting transactional ownership."
- "Commit job intent through an outbox and make worker publication idempotent."
- "Rollback is simple only while data ownership and compatibility remain controlled."
- "The facade routes on cached assignments; a control-plane outage never touches the request path."
- Wrap-up script
 - "The design keeps one transactional owner, adds a routing seam, and moves report execution to workers fed by a transactional outbox."
 - "The key invariant is exactly one authoritative writer per entity, so rollback stays a routing change with no data movement."
 - "The main tradeoff is at-least-once delivery, which buys durability across a queue outage at the price of idempotent claims and conditional publication."

- “My weakest assumption is that report CPU, not the database, is the bottleneck; I would confirm it with a profile before touching the seam.”

References

- [Ranked question and report provenance](#).
- [OCI role framing](#).
- [Detailed API and schema companion](#).
- Reference check: 2026-09-09.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values and architecture choices remain proposed interview assumptions.
- [Strangler fig pattern](#): gradual migration through a routing boundary.
- [Transactional outbox](#): atomic intent and duplicate delivery.
- [Database connection-pool reference](#): retained LLD background for pool sizing.